Lecture 3
Shell

Comp 206 – Introduction to Software Systems – Fall 2026

Mathieu Bérubé-Vallières

McGill University

September 8, 2026

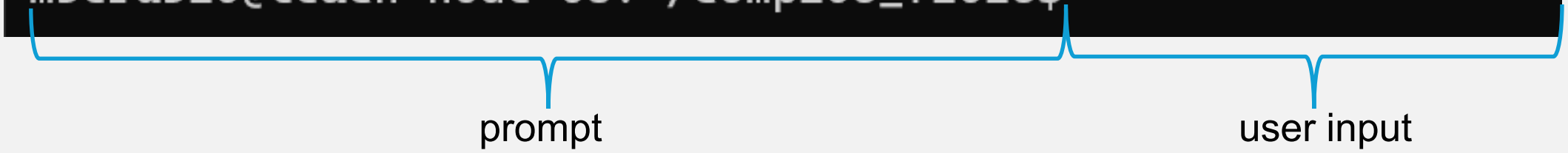
Shell

- The *shell* is a program that provides users access to the system on which it runs
  - It provides a way to get user input (i.e. the *command-line*), displays OS information, and stores session information (e.g. the previous commands you ran)
- There are many shells
  - sh, bash (most common Linux shell), zsh (default for macOS), PowerShell, cmd.exe
  - We will use Bash (which is the default on mimi)
- The *terminal* emulator lets you send keyboard input to the shell and displays text from the shell

Bérubé-Vallières © 2026
See last slide for complete credits

Command-line interface

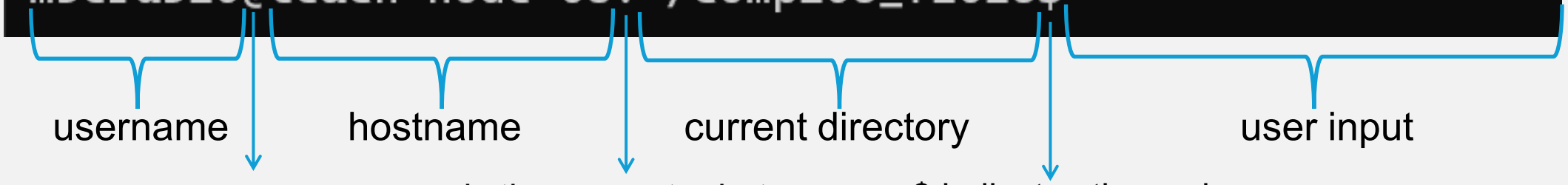
```
mberub20@teach-node-08:~/comp206_f2026$ ls
demo  demo_2  hello.txt
mberub20@teach-node-08:~/comp206_f2026$ ls -l demo
total 2
drwxr-xr-x 2 mberub20 nogroup 5 Sep  3 15:52 comp206
-rw-r--r-- 1 mberub20 nogroup 0 Sep  3 15:50 foo.txt
-rw-r--r-- 1 mberub20 nogroup 0 Sep  3 15:52 hello.txt
mberub20@teach-node-08:~/comp206_f2026$
```



Bérubé-Vallières © 2026
See last slide for complete credits

Command-line interface

```
mberub20@teach-node-08:~/comp206_f2026$ ls
demo  demo_2  hello.txt
mberub20@teach-node-08:~/comp206_f2026$ ls -l demo
total 2
drwxr-xr-x 2 mberub20 nogroup 5 Sep  3 15:52 comp206
-rw-r--r-- 1 mberub20 nogroup 0 Sep  3 15:50 foo.txt
-rw-r--r-- 1 mberub20 nogroup 0 Sep  3 15:52 hello.txt
mberub20@teach-node-08:~/comp206_f2026$
```



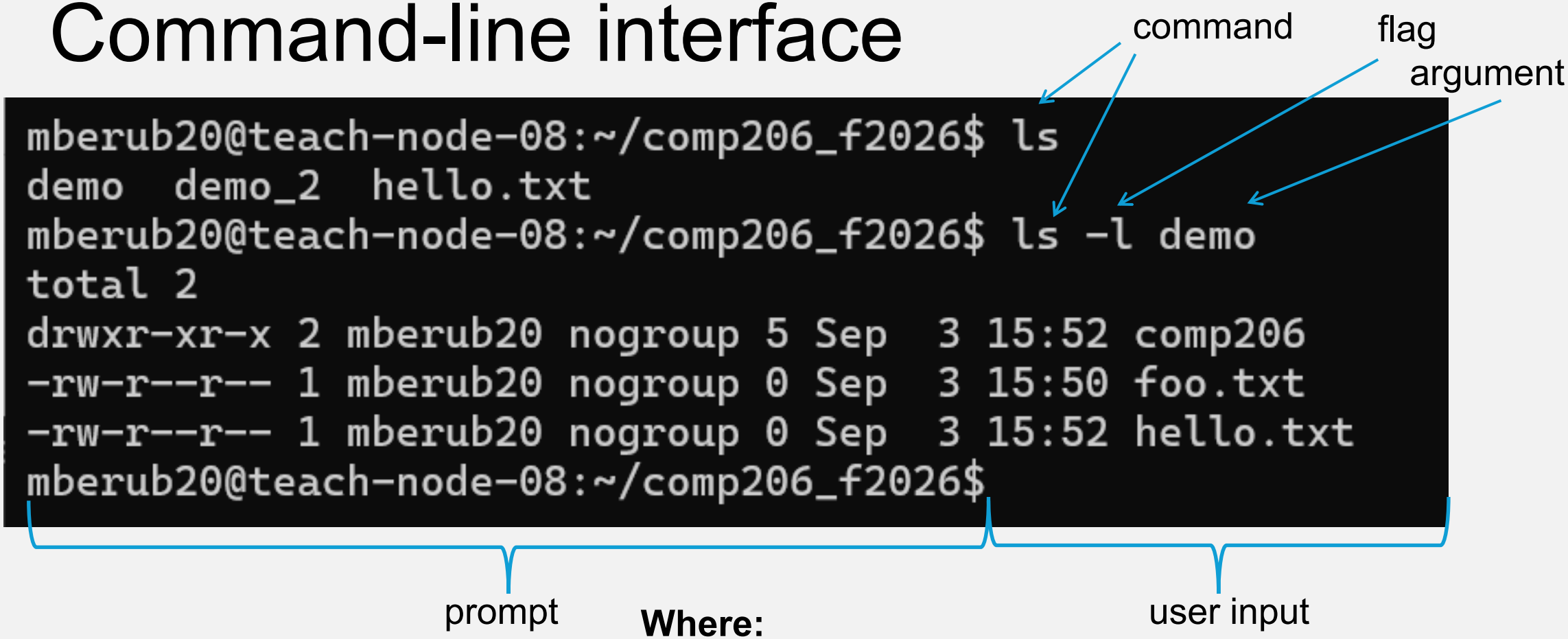
@ is the separator between username and hostname

: is the separator between user and current directory

$ indicates the end of the prompt and the start of the user input

Bérubé-Vallières © 2026
See last slide for complete credits

Command-line interface



Input format:
Command flags arguments

(order of flags and arguments is not always strict)

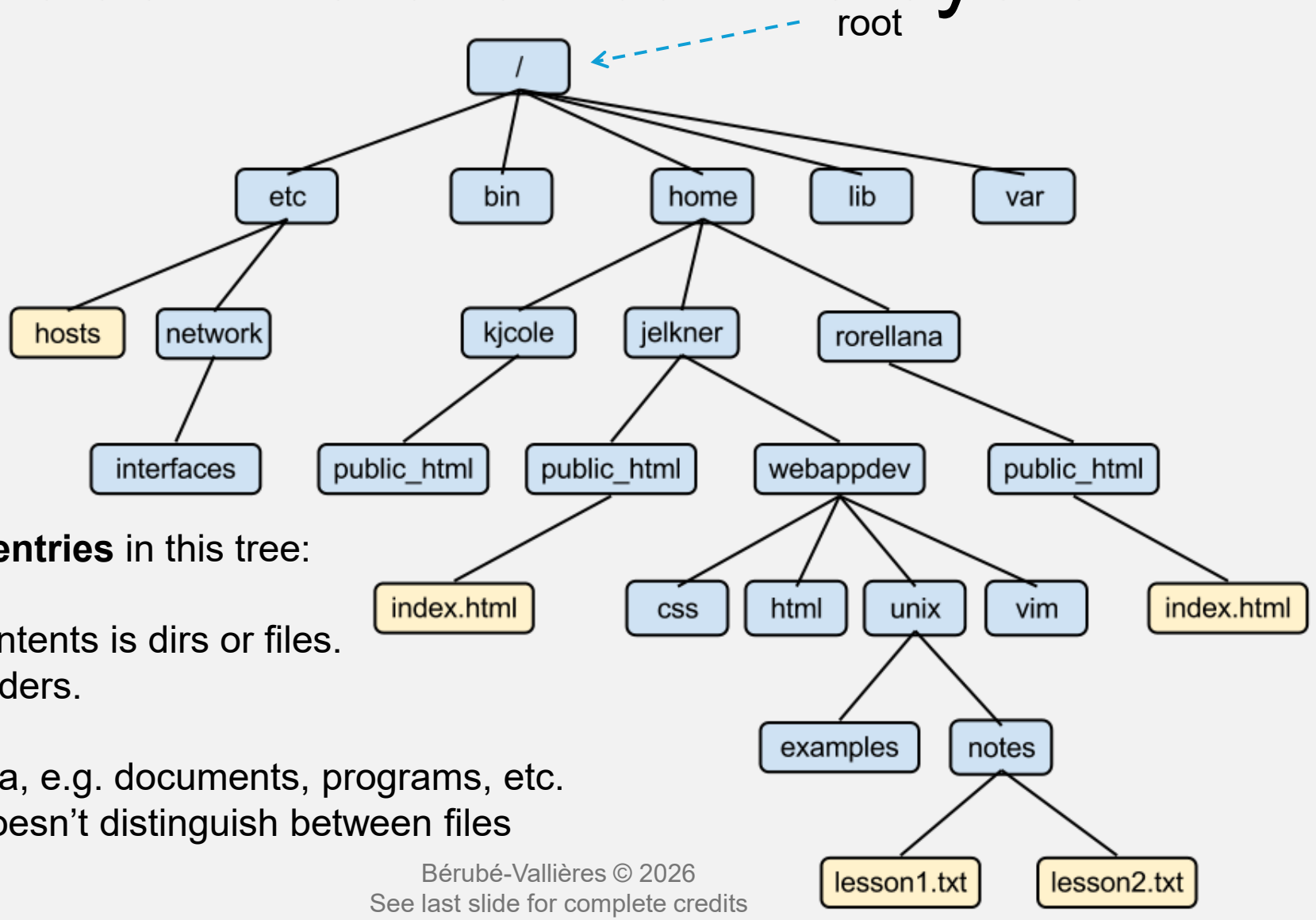
Where:

- Command: The program to run
- Flags (also called switches or options): modifies behaviour of the command
- Arguments (also called parameters): input passed to the command

Last class: File System (FS)

Bérubé-Vallières © 2026
See last slide for complete credits

Last class: Hierarchical file system:



Two different kind of **entries** in this tree:

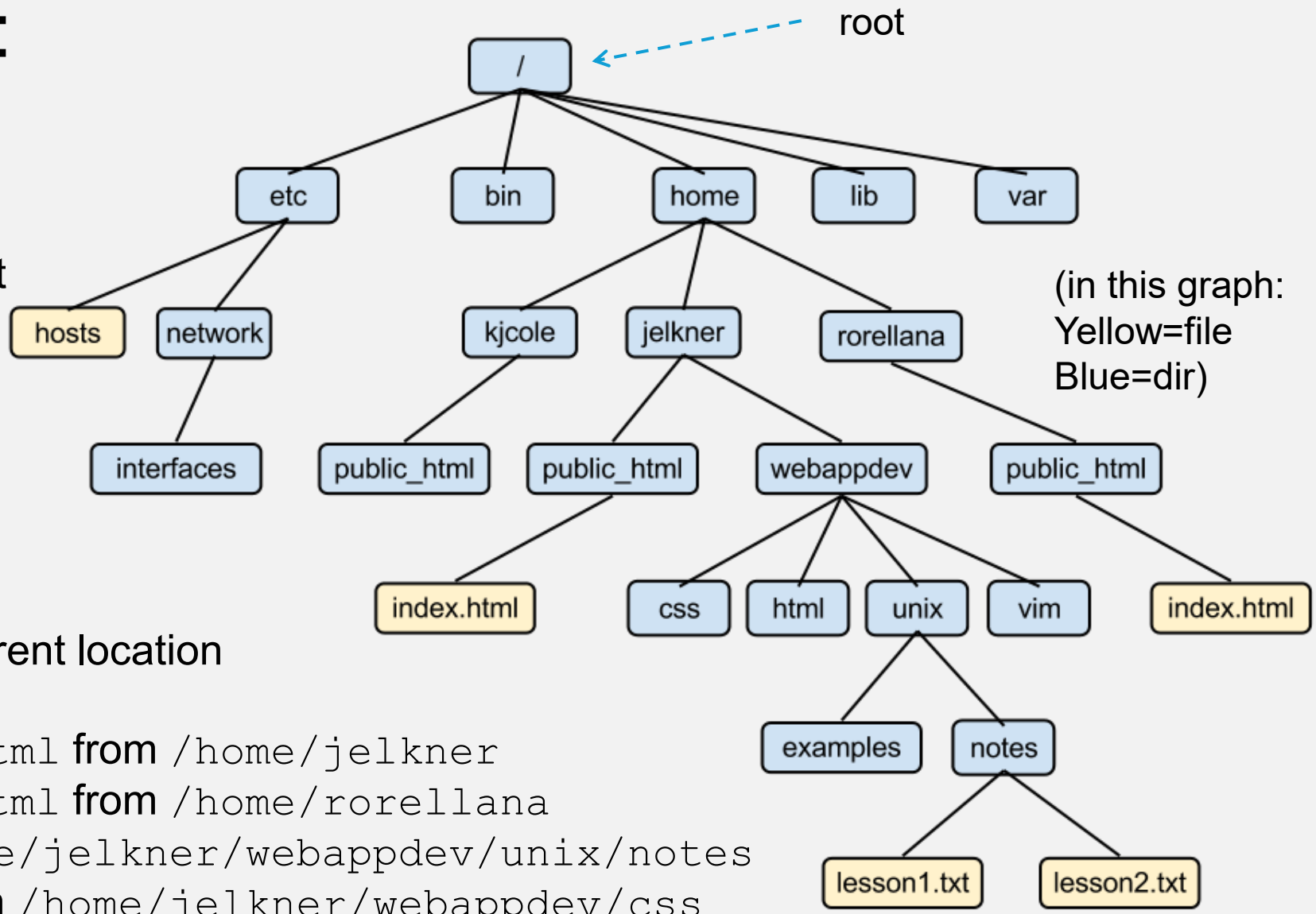
- **Directories**
  - Structural. Contents is dirs or files. Also called folders.
- **Files**
  - Content is data, e.g. documents, programs, etc. File system doesn't distinguish between files

Bérubé-Vallières © 2026
See last slide for complete credits

Last time: File/dir location: paths

Absolute path:
A string that begins at the root

Examples:
- `/etc/hosts`
- `/home/jelkner`

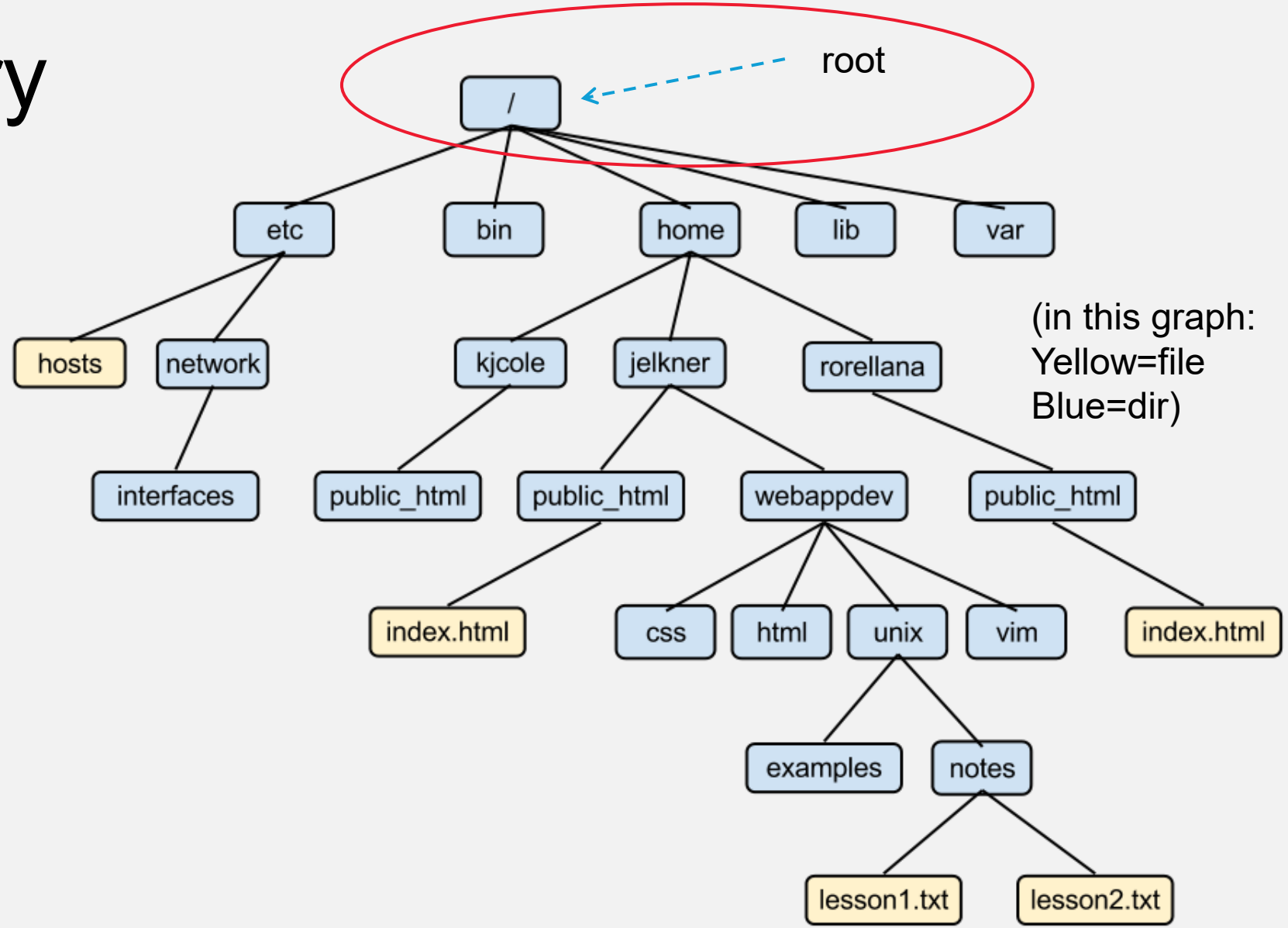


Relative path:
A string that begins at the current location
Examples:
- `public_html/index.html` from `/home/jelkner`
- `public_html/index.html` from `/home/rorellana`
- `lesson1.txt` from `/home/jelkner/webappdev/unix/notes`
- `../unix/examples` from `/home/jelkner/webappdev/css`

Bérubé-Vallières © 2026
See last slide for complete credits

Root directory

- The root is the top folder of the OS
- Special symbol: /



Bérubé-Vallières © 2026
See last slide for complete credits

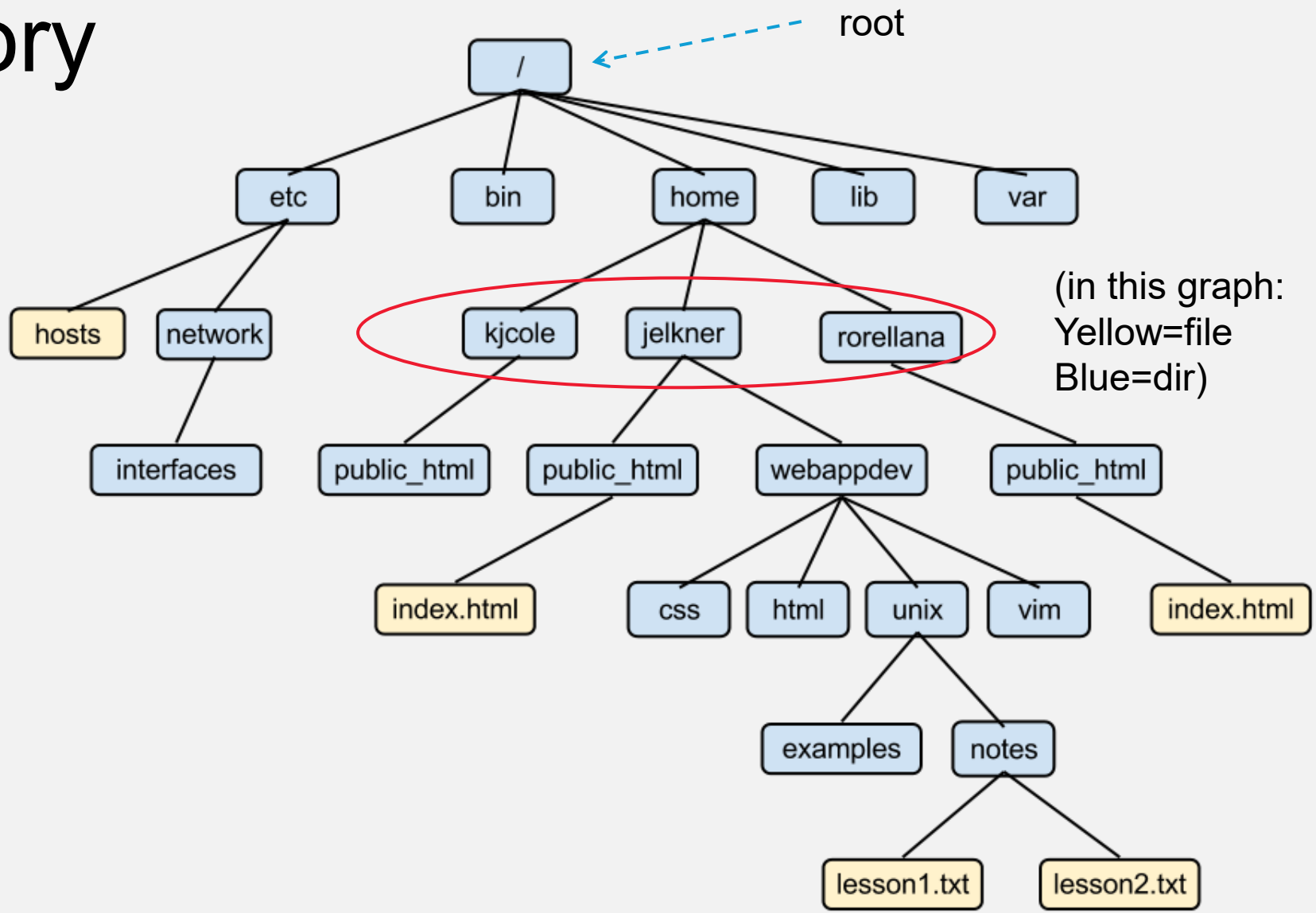
Home directory

- The home is the top folder in the user's directory tree
- Special symbol: `~`

(we will see later that this is a *wildcard* that gets *expanded* to the absolute path of the user's home)

If user `kjcole` types `~`, this refers to `/home/kjcole`

If user `jelkner` types `~`, this refers to `/home/jelkner`



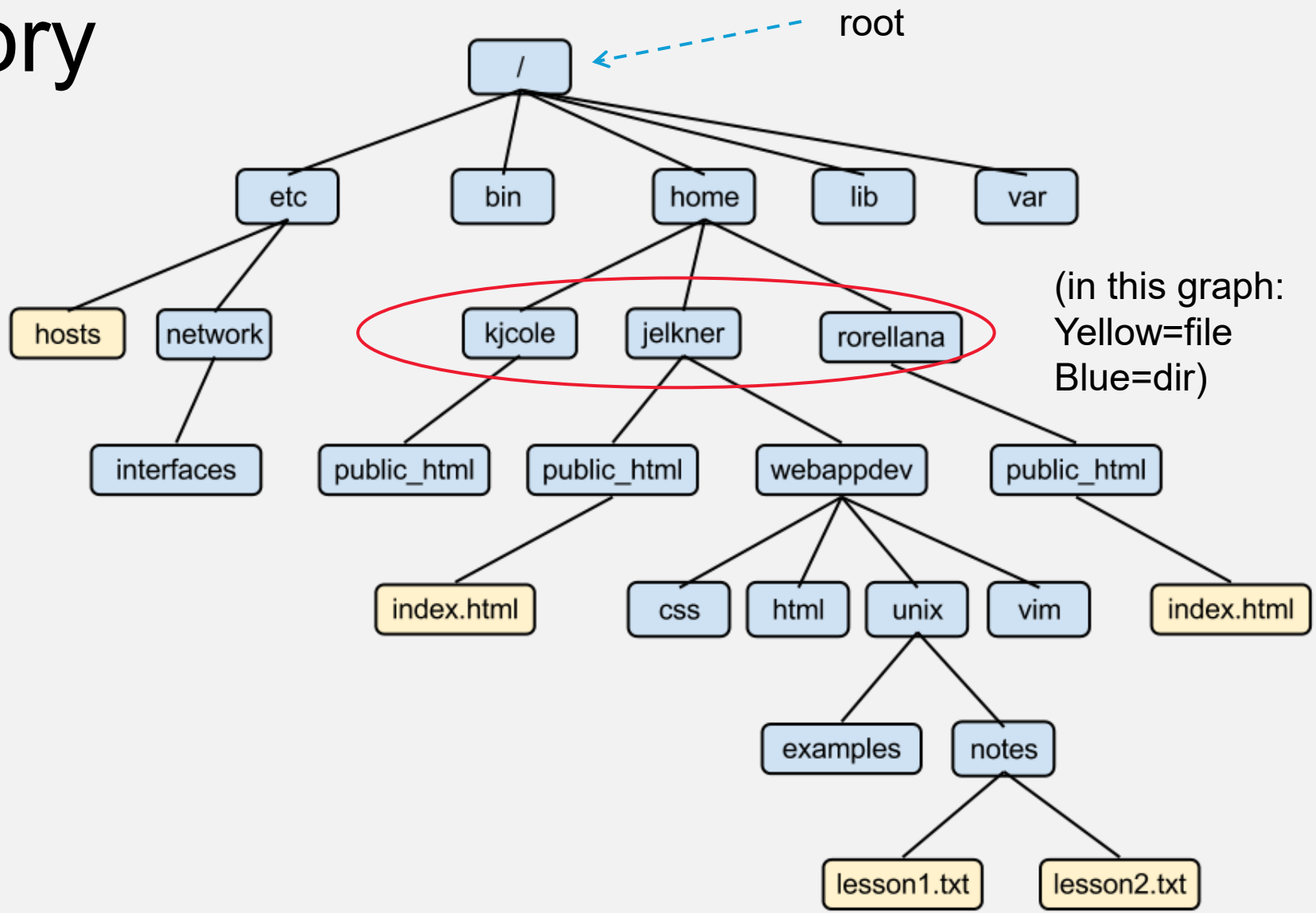
Bérubé-Vallières © 2026
See last slide for complete credits

Home directory

- The home is the top folder in the user's directory tree
- Special symbol: `~`

(we will see later that this is a *wildcard* that gets *expanded* to the absolute path of the user's home)

Exercise: What is your home on mimi? Type `echo ~` to find out!

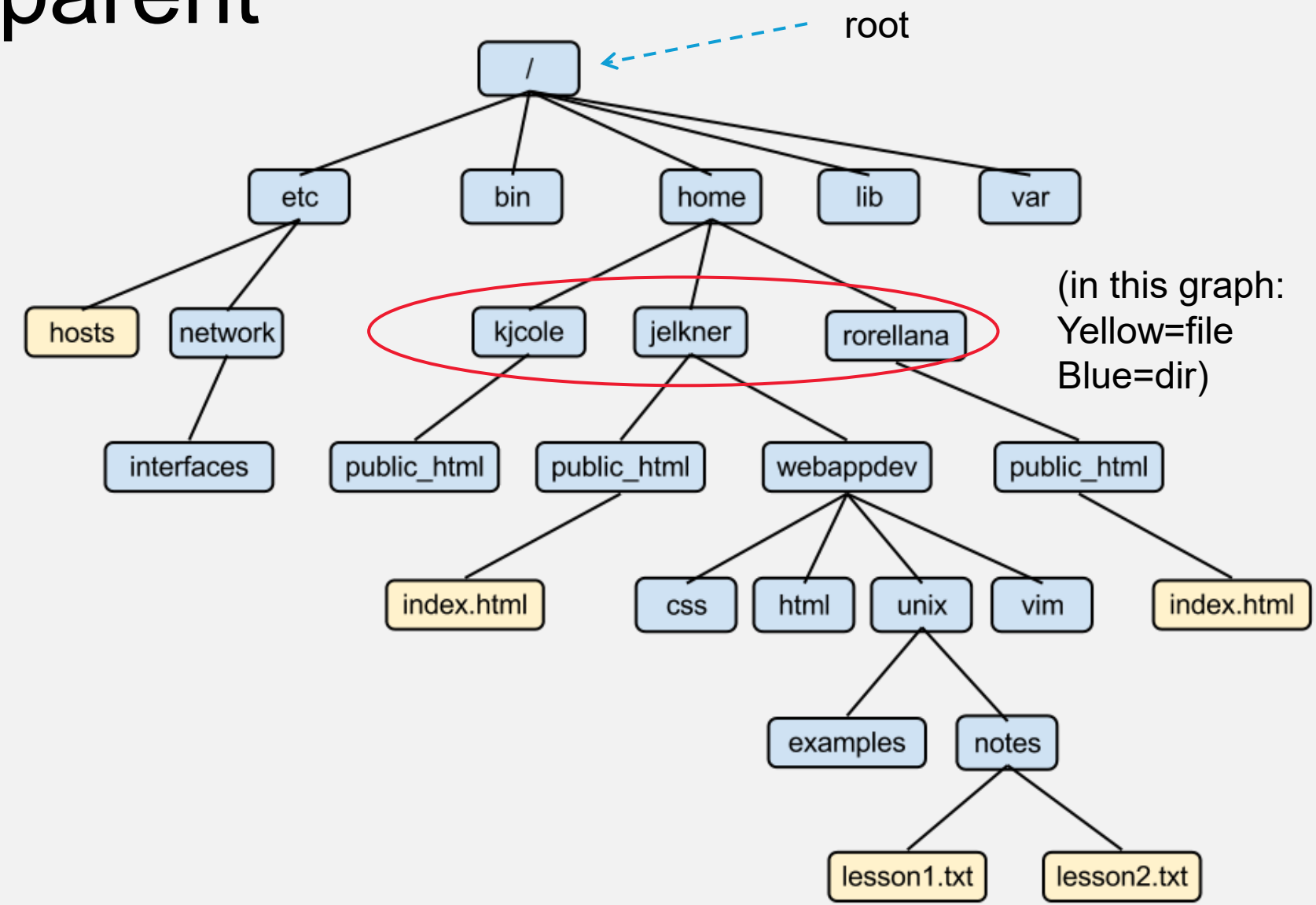


Bérubé-Vallières © 2026
See last slide for complete credits

Current and parent directories

- The current directory (also called working directory) has the following special symbol: `.`
- The parent directory (the directory "above" the current directory) has the following special symbol: `..`

Note: We will see later that these are not wildcards: they are hardcoded entries (which you can see with `ls -a`)



Bérubé-Vallières © 2026
See last slide for complete credits

The truth about Unix files and directories

- Philosophy: Everything is a file
- Directories are simply a special type of file that contains a list of index nodes (*inodes*)
  - Index nodes contain metadata about a file, including whether it's a regular file or directory (or other special types), who owns it, the permissions, etc., and the location of the file on the disk
  - Will not discuss inodes much, but good to keep in mind
    - We will sometimes use "regular file" when we want to talk specifically about "standard" files
- Unix does not impose any file structure (file format) to regular files. Their structure and the way to interpret them is entirely dependent on the software using them.
  - File extensions mean nothing and are *only* useful for the user. Different from Windows!

Bérubé-Vallières © 2026
See last slide for complete credits

First UNIX file manipulation commands

Bérubé-Vallières © 2026
See last slide for complete credits

`ls`

- Description: **Lis**t directory content
  - List information about the FILEs (current directory by default). Sort alphabetically by default
    - Note that FILE could be regular file or directory
- Syntax: `ls [OPTION]… [FILE]…`
  - brackets means optional, and … means there may be multiple
- Common flags (options):
  - -l: see contents in long format
  - -a: do not ignore hidden files (files that start with .)
  - Exercise: Find how to use different sort using **man ls**

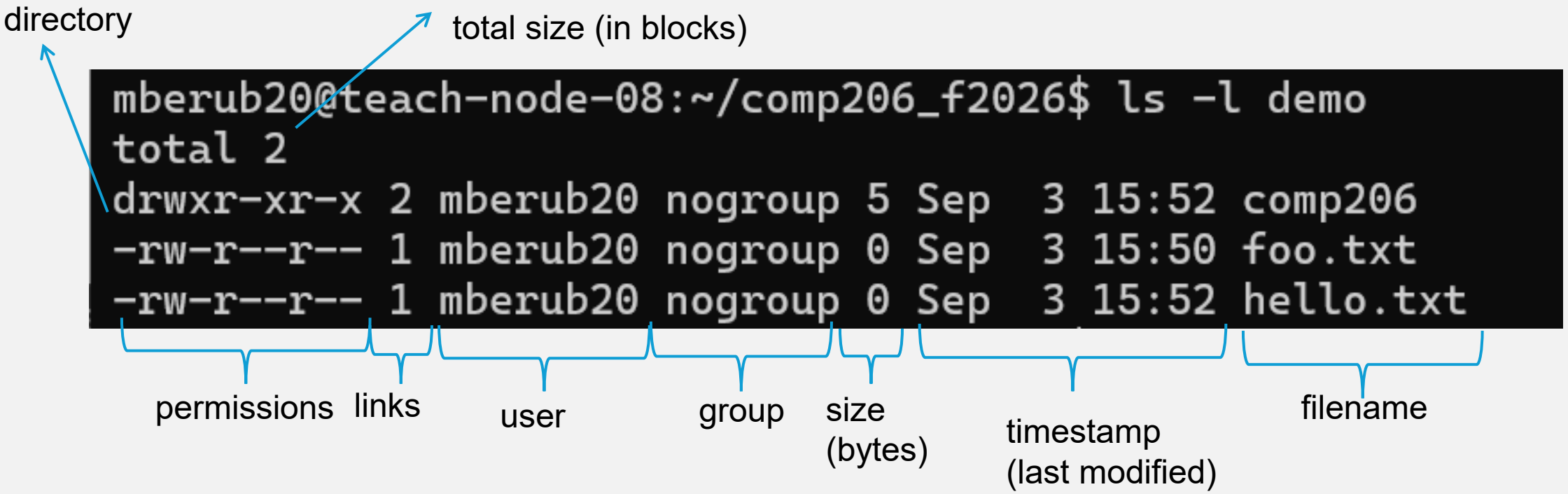
Bérubé-Vallières © 2026
See last slide for complete credits

`ls` (examples)

- List contents of current directory:
  - `ls`
- List contents of current directory and include hidden files (files that start with . are *hidden files*)
  - `ls -a`
- List contents of `/etc/network`
  - `ls /etc/network`
  - `ls network` (if currently in /etc)
- List contents of `/etc/network` in *long format*
  - `ls -l /etc/network`
- What happens when using with a regular file?
  - Can be useful to see more information about a file!

Bérubé-Vallières © 2026
See last slide for complete credits

`ls` (long format)



Bérubé-Vallières © 2026
See last slide for complete credits

`mkdir`

- Description: **Ma**ke **dir**ectories
  - Create directories if they do not already exist
- Syntax: `mkdir [OPTION]… DIRECTORY…`

Bérubé-Vallières © 2026
See last slide for complete credits

`touch`

- Description: Makes an empty file
  - The original purpose of touch is to update a file timestamp, but a side-effect is that it creates the file if it does not exist, which became its most common uses
- Syntax: `touch [OPTION] FILE`

Bérubé-Vallières © 2026
See last slide for complete credits

`rm`

- Description: **Rem**ove files or directories
- Syntax: `rm [OPTION]… [FILE]…`
- Common flags:
  - -r or -R: recursively visits a directory, first visiting the files and subdirectories beneath it
    - necessary to delete directories!
- Important: No recycle bin!

Bérubé-Vallières © 2026
See last slide for complete credits

mv

- Description: **Mov**e files or directories
- Syntax:
  - `mv [OPTION]… SOURCE DEST`
  - `mv [OPTION]… SOURCE… DIRECTORY`
- Move source to destination, or source(s) to directory
  - If the last argument is a directory, will move files to that directory!
  - Otherwise, it moves and rename the file/directory
- Important: Does not ask before overwriting!

Bérubé-Vallières © 2026
See last slide for complete credits

cp

- Description: **Copy** files or directories
- Syntax:
  - `cp [OPTION]… SOURCE DEST`
  - `cp [OPTION]… SOURCE… DIRECTORY`
- Copy source to destination, or source(s) to directory
  - If the last argument is a directory, will copy files to that directory!
  - Otherwise, it copies and rename the file/directory
- Important: Does not ask before overwriting!
- Common flags:
  - -r or -R: recursively visits a directory, first visiting the files and subdirectories beneath it
    - necessary to copy directories!

Bérubé-Vallières © 2026
See last slide for complete credits

`cat` and `more`

- `cat [OPTIONS]… FILE…`

– file concatenate and display the concatenated result

- `more [OPTIONS]… FILE…`

– page through a text file

Bérubé-Vallières © 2026
See last slide for complete credits

`man`

- The man command allows you to access the on-line manual pages of the various commands available on the shell.
  - These pages are often referred to as “man pages”.
- The man pages are your first source of information when working in the shell.
- To access a man page, simply type man and the name of the command at the prompt.

- `man ls`

Bérubé-Vallières © 2026
See last slide for complete credits

Additional credits

- Some content from this lecture is based on:
  - Past Comp 206 slides from Prof. Vybihal, Kopinsky and Errington

Bérubé-Vallières © 2026
See last slide for complete credits